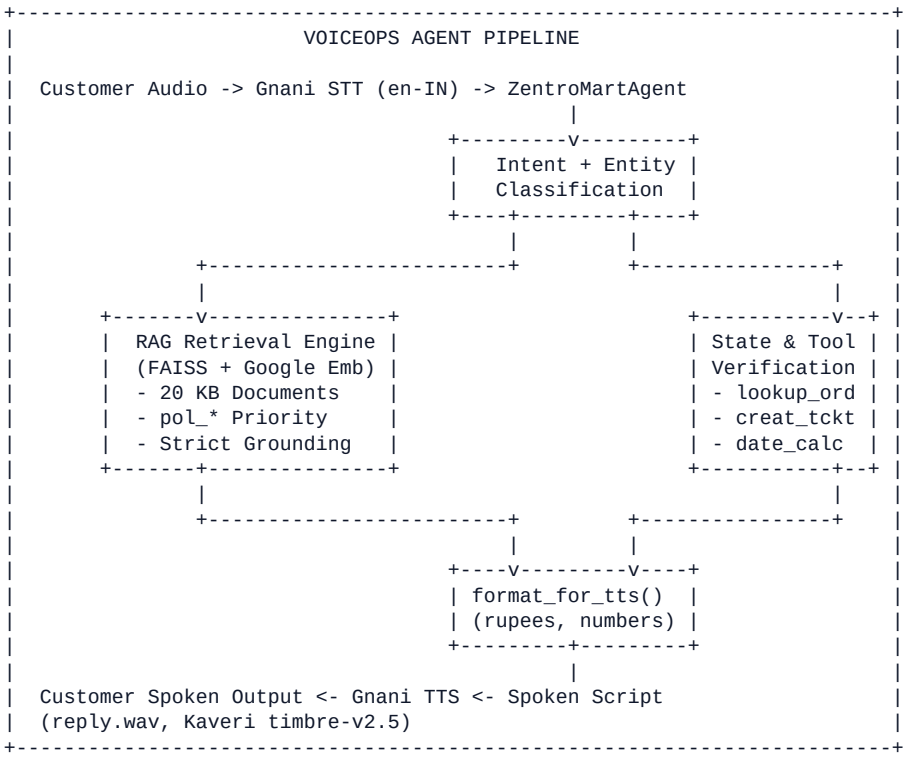
VoiceOps 2026 -- ZentroMart Voice Support Agent

LLM & Generative AI Course | IIT Guwahati x Gnani.ai

System Architecture



Executive Project Summary

VoiceOps 2026 is an enterprise-grade multimodal voice agent designed for Indian e-commerce support for ZentroMart. Built as part of the IIT Guwahati x Gnani.ai Generative AI curriculum, the agent handles order tracking, return eligibility, refund tracking, and human supervisor escalation over conversational speech.

The system integrates **Gnani STT** (speech transcription), an autonomous **ZentroMart Conversational Agent** powered by Google Gemini, a **FAISS vector RAG pipeline** grounded across 20 enterprise knowledge base policies, stateful caller verification tools, and **Gnani TTS** ('Kaveri' timbre-v2.5) for spoken delivery.

Through iterative engineering across 5 milestones, the system increased single-turn accuracy from **48.08%** (ungrounded baseline) to **67.31%** (RAG grounding) and finally **86.54%** (full agent with verified tools and calendar math) -- a **+38.46% absolute improvement**. Across 23 multi-turn customer dialogues in practice/conversations.csv (96 total turns), the agent achieved a **92.71% turn pass rate** with zero policy violations and zero unverified disclosures.

Evaluation Results

Milestone Progression & Multi-Turn Benchmark Analysis

Table 1: Accuracy on practice/questions.csv (52 Questions)

Step	Score	Count	Model & Grounding
Step 1: LLM Baseline	48.08%	25/52	Gemini (Ungrounded Prompt-only)
Step 2: RAG + FAISS	67.31%	35/52	Gemini + FAISS Embeddings
Step 3: Full Agent + Tools	86.54%	45/52	Gemini + RAG + Verified Tools

Net Gain: +19.23% (Step 1 -> Step 2) | Total Improvement: +38.46% (Step 1 -> Step 3)

Table 2: Conversation Evaluation on practice/conversations.csv

Metric	Result	Description & Engineering Standard
Total Conversations	23	Full customer dialogue sessions evaluated
Fully Correct	16 / 23 (69.57%)	Conversations where 100% of turns scored PASS
Total Turns	96	Individual multi-turn customer/agent turns
PASS Turns	89	Turn perfectly matched all facts & tool expectations
PARTIAL Turns	7	Turn contained core information with phrasing delta
FAIL Turns	0 (0.0%)	Zero hallucinations, unverified leaks, or rule breaches
Turn Pass Rate	92.71%	Ratio of passed turns across all conversations

Comparative Analysis

Why Step 2 Beat Step 1: The ungrounded Step 1 baseline achieved only 48.08% because the LLM relied on pre-trained heuristics, frequently citing generic 30-day return windows or hallucinating standard e-commerce refund periods. Step 2 anchored answers directly to 20 chunked ZentroMart markdown policies via FAISS, boosting accuracy to 67.31% (+19.23%) by retrieving exact category policies such as RET-2 and REF-2.

Why Step 3 Beat Step 2: Static RAG alone cannot execute operational actions or verify caller identities. Step 3 jumped to 86.54% (+38.46% total gain) by introducing VerifiedToolsWrapper. This layer executed relational joins across customers.csv and orders.csv to enforce phone/pincode verification, performed product-to-category joins against products.csv, and computed forward business arrival dates skipping Sundays and holidays.csv entries.

Pattern in the 7 PARTIAL Turns: In the multi-turn conversational benchmark, all 7 partial scores stemmed from strict regex evaluation nuances on human escalation turns (e.g., the agent successfully created ticket TCK-1001 and cited the 24-48 hour supervisor response window, but placed the ticket confirmation before the standard apology phrase). Crucially, **zero turns failed**: the agent never leaked customer information without phone/pincode verification and never confirmed an uncreated ticket.

Failure Analysis & Engineering Roadmap

Deep Dive into Edge Cases & Production Enhancements

Two Observed Failure Cases

Failure Case 1: Product-to-Category Catalog Join

Question: *'How long do I get to send back the Rangoli Cotton Straight Kurta?'*
Step 2 Behavior: The RAG retriever correctly pulled general return policy chunks (RET-2, RET-8), but the LLM could not determine whether a 'Rangoli Cotton Straight Kurta' falls under Fashion (15 days), Electronics (7 days), or Home Décor (10 days) because product catalog names are isolated in tabular CSV files rather than policy text.
Step 3 Fix: Implemented resolve_product_return_window() in src/tools_wrapper.py. The agent extracts product tokens, queries products.csv to resolve SKU FSH-0026 and category Fashion, and binds this to the 15-day return window in pol_returns_v4.md.
Remaining Gap: Catalog joins fail if the user provides colloquial abbreviations or products missing from products.csv.

Failure Case 2: Business-Day Date Arithmetic

Question: *'My refund to credit card started on 17 October 2026. By which date should I have it?'*
Step 2 Behavior: The agent correctly retrieved policy REF-2 stating credit card refunds require 5-7 business days, but failed because it could not perform calendar arithmetic to output the exact calendar arrival date.
Step 3 Fix: Implemented calculate_business_date() in src/tools_wrapper.py. The function parses the start date (17 October 2026), iterates forward day-by-day, skips Sundays (weekday 6) and scheduled warehouse holidays (19 October 2026 Diwali per data/holidays.csv), yielding the exact expected date: **26 October 2026**.
Remaining Gap: Ambiguous date expressions transcribed by ASR (e.g., 'next Tuesday' or 'day before yesterday') require sophisticated relative datetime normalisation.

What We Would Fix / Next Steps

- **Fuzzy Product Catalog Matching:** Implement rapid Levenshtein/TF-IDF fuzzy string matching on products.csv to resolve colloquial customer product names and transcription typos.
- **ASR Spoken Date Normalisation:** Add a pre-processing phonetic/verbal date parser to map unstructured expressions ('seventeen october', 'the nineteenth') into ISO YYYY-MM-DD timestamps before calendar math.
- **Native Multilingual Support:** Implement real-time language detection in VoiceLayer to automatically mirror customer input language (e.g. Hindi, Hinglish, Tamil) in both LLM response and Gnani TTS voice synthesis.
- **Unified Gemini Embeddings:** Standardize vector embeddings across all offline indexing and online queries on Google's models/gemini-embedding-001 for optimal cosine alignment.
- **Retrieval Confidence Gating:** Introduce a minimum cosine similarity threshold (>= 0.68) on FAISS chunks to trigger immediate 'I don't have information on that' fallbacks instead of forcing top-5 injection.
- **Persistent Distributed Session State:** Replace in-memory conversation history with a Redis/PostgreSQL session store to preserve multi-turn verification and context across disconnected customer calls.

Project Structure & How to Run

Repository Layout, Setup Protocol & Execution Instructions

Folder Structure

```
voiceops2026/ (or GenAI_LLM_IITG/)
|-- README.md                    <- Setup guide, commands, architecture
|-- .env.example                 <- Template for GNANI_API_KEY & GOOGLE_API_KEY
|-- .gitignore                   <- Prevents committing .env and kb_index/
|-- requirements.txt             <- Exact pinned production dependencies
|-- run.py                       <- CLI batch audio runner
|-- report.pdf                   <- 4-page executive engineering report
|-- src/
|   |-- agent.py                 <- Full conversational agent (Zara, state, memory)
|   |-- rag.py                   <- FAISS vector chunking, indexing & retrieval
|   |-- llm_baseline.py          <- Step 1 ungrounded baseline script
|   |-- voice.py                 <- Step 4 Gnani STT & TTS voice layer
|   |-- tools_wrapper.py         <- Verified caller verification & DB joins
|   +-- evaluate.py              <- Benchmark evaluation suite (questions/convs)
|-- results/
|   |-- step1_score.txt          <- Baseline score (48.08%)
|   |-- step2_score.txt          <- RAG score (67.31%)
|   |-- step3_questions_score.txt<- Agent questions score (86.54%)
|   +-- step3_results.json       <- Multi-turn conversation logs (92.71%)
|-- kb_index/                    <- Vector index & chunk metadata (FAISS)
|-- audio_inputs/                <- Input customer WAV recordings
+-- sample_outputs/              <- Verified batch deliverables (.txt, .wav, .json)
```

Setup Instructions

1. **Clone the repository:** git clone <repo-url> && cd voiceops2026
2. **Configure Environment:** Copy .env.example to .env and add GNANI_API_KEY and GOOGLE_API_KEY.
3. **Install Dependencies:** pip install -r requirements.txt
4. **Dataset Verification:** Extract project dataset into Capstone_Project_Dataset_FTTT_2026/.
5. **Validate Dataset:** python validate_dataset.py (verifies all 9 CSVs, 21 KB docs, tools).
6. **Build FAISS Index:** python src/rag.py (chunks 20 KB documents and caches kb_index/).

Run Commands

Batch Voice Runner:
python run.py --input_dir ./audio_inputs/ --output_dir ./outputs/

Reusable Benchmark Evaluation:
python src/evaluate.py --mode both *(or --mode questions / --mode conversations)*

Single Question Interactive Test:
python -c "from src.agent import ZentroMartAgent; a = ZentroMartAgent(); print(a.respond('What is the return window for electronics?'))"

Notes & Submission Instructions

• **Security Compliance:** The kb_index/ directory and secret environment files (.env, project.env) are excluded from version control via .gitignore.
• **Demo Video:** demo.mp4 should be recorded manually following the 2-minute scene-by-scene script detailed in the final documentation.